CSE204 – Database Systems

Group Project – Assignment 1

Airbnb-Style Property Rental Management System

Database Design & Normalization Report

March 2026

Team Member 1: Ömer Özenç
Team Member 2: Erkan Ceylan
Team Member 3: Emirhan Sak
Team Member 4: Yasin Taha İnal
Team Member 5: Muhammed Can Akbaş

1. Project Description

This report presents the design and implementation of a relational database system for an Airbnb-style Property Rental Management System — an online marketplace where hosts list properties for short-term rental, and guests can search, book, and review them.

The system manages:

- Users – registration and account management for all participants
- Hosts – users who list one or more properties
- Guests – users who search for and book accommodations
- Properties – listings with location, pricing, capacity, and amenities
- Bookings – reservation records with dates and guest counts
- Payments – financial transactions tied to bookings
- Reviews – guest feedback and ratings for completed stays
- Amenities – features offered by properties (Wi-Fi, pool, parking, etc.)

The database stores, organizes, and retrieves all platform data while ensuring integrity, minimizing redundancy, and supporting operational needs.

2. Data Items (Attributes)

A data item (attribute) is the smallest unit of data that has meaning within a given context — corresponding to a column in a table. The following 59 data items are maintained across 10 entities:

#	Data Item	Description	Data Type	Table
1	user_id	Unique identifier for each user	INT	Users
2	full_name	Full name of the user	VARCHAR(100)	Users
3	email	Email address (unique)	VARCHAR(150)	Users
4	phone	Phone number	VARCHAR(20)	Users
5	password_hash	Hashed password for auth	VARCHAR(255)	Users
6	account_created_at	Timestamp of account creation	DATETIME	Users
7	account_status	Status: active/suspended/deleted	VARCHAR(20)	Users
8	host_id	Unique identifier for each host	INT	Hosts
9	user_id (FK)	Reference to the Users table	INT	Hosts
10	host_since	Date the user became a host	DATE	Hosts
11	host_status	Status: active/inactive	VARCHAR(20)	Hosts
12	guest_id	Unique identifier for each guest	INT	Guests
13	user_id (FK)	Reference to the Users table	INT	Guests
14	nationality	Guest nationality	VARCHAR(50)	Guests
15	date_of_birth	Guest date of birth	DATE	Guests
16	property_id	Unique identifier for each property	INT	Properties
17	host_id (FK)	Reference to Hosts table	INT	Properties
18	title	Listing title	VARCHAR(200)	Properties
19	description	Property description	TEXT	Properties
20	property_type	Type: apartment/house/villa/room	VARCHAR(50)	Properties
21	city	City of property	VARCHAR(100)	Properties
22	district	District/neighborhood	VARCHAR(100)	Properties
23	address_line	Full street address	VARCHAR(255)	Properties
24	nightly_price	Price per night (USD)	DECIMAL(10,2)	Properties

#	Data Item	Description	Data Type	Table
25	cleaning_fee	One-time cleaning fee	DECIMAL(10,2)	Properties
26	capacity	Max number of guests	INT	Properties
27	bedroom_count	Number of bedrooms	INT	Properties
28	bathroom_count	Number of bathrooms	INT	Properties
29	photo_id	Unique identifier for each photo	INT	PropertyPhotos
30	property_id (FK)	Reference to Properties table	INT	PropertyPhotos
31	file_name	Image file name	VARCHAR(255)	PropertyPhotos
32	file_path	Storage path of image	VARCHAR(500)	PropertyPhotos
33	caption	Photo description	VARCHAR(200)	PropertyPhotos
34	display_order	Display order of photos	INT	PropertyPhotos
35	booking_id	Unique identifier for each booking	INT	Bookings
36	property_id (FK)	Reference to Properties table	INT	Bookings
37	guest_id (FK)	Reference to Guests table	INT	Bookings
38	booking_date	Date booking was made	DATE	Bookings
39	check_in_date	Check-in date	DATE	Bookings
40	check_out_date	Check-out date	DATE	Bookings
41	guest_count	Number of guests	INT	Bookings
42	total_price	Total price for the stay	DECIMAL(10,2)	Bookings
43	booking_status	Status: confirmed/cancelled/completed	VARCHAR(20)	Bookings
44	payment_id	Unique identifier for each payment	INT	Payments
45	booking_id (FK)	Reference to Bookings table	INT	Payments
46	payment_date	Date of payment	DATE	Payments
47	amount	Payment amount (USD)	DECIMAL(10,2)	Payments
48	payment_method	Method: credit_card/paypal/bank_transfer	VARCHAR(30)	Payments
49	payment_status	Status: pending/completed/refunded	VARCHAR(20)	Payments
50	review_id	Unique identifier for each review	INT	Reviews
51	booking_id (FK)	Reference to Bookings table	INT	Reviews
52	rating	Rating (1-5)	INT	Reviews
53	comment	Text of the review	TEXT	Reviews
54	review_date	Date the review was posted	DATE	Reviews
55	amenity_id	Unique identifier for each amenity	INT	Amenities
56	amenity_name	Name of the amenity	VARCHAR(100)	Amenities
57	amenity_category	Category: basic/comfort/safety/outdoor	VARCHAR(50)	Amenities
58	property_id (FK)	Reference to Properties table	INT	PropertyAmenities
59	amenity_id (FK)	Reference to Amenities table	INT	PropertyAmenities

3. Unnormalized Form (UNF)

In the Unnormalized Form, all data is stored in a single flat table with no regard for redundancy or repeating groups. Host names, guest details, property information, amenities, and photos are all combined into one row.

3.1 UNF Table Structure

The UNF table contains 39 columns, including repeating group columns (amenity_1, amenity_2, amenity_3, photo_1, photo_2) that violate atomicity. Below is a representative subset of the sample data (15 rows total):

booking_id	booking_date	check_in	guest_id	guest_name	property_id	property_title	city	host_name	payment_method	review_rating
1	2025-06-01	2025-06-15	1	Alice Johnson	101	Cozy Downtown Apt	Istanbul	Mehmet Yilmaz	credit_card	5
2	2025-06-05	2025-06-20	2	Bob Smith	102	Sea View Villa	Antalya	Ayse Demir	paypal	4
3	2025-06-10	2025-07-01	3	Carla Martinez	101	Cozy Downtown Apt	Istanbul	Mehmet Yilmaz	credit_card	NULL
4	2025-06-12	2025-07-05	1	Alice Johnson	103	Mountain Retreat	Bolu	Can Ozturk	bank_transfer	5
5	2025-06-15	2025-07-08	4	David Lee	102	Sea View Villa	Antalya	Ayse Demir	credit_card	3

Note: Columns omitted for readability. Full 15-row dataset with all 39 columns is in the SQL INSERT statements (Section 5). Notice the significant redundancy — host and property data (e.g., 'Mehmet Yilmaz', 'Cozy Downtown Apt') repeats across multiple booking rows.

3.2 Problems in UNF

- Repeating groups: amenity_1, amenity_2, amenity_3 and photo_1, photo_2 are multi-valued
- Data redundancy: host and property details repeat for every booking
- Update anomaly: changing a host's email requires updating many rows
- Deletion anomaly: deleting a booking could lose guest or property data
- Insertion anomaly: a property cannot be added without a booking

4. Functional Dependencies

A functional dependency (FD) exists when the value of one attribute uniquely determines the value of another. These are the foundation of the normalization process.

Key functional dependencies:

- user_id → full_name, email, phone, password_hash, account_created_at, account_status
- host_id → user_id, host_since, host_status
- guest_id → user_id, nationality, date_of_birth
- property_id → host_id, title, description, property_type, city, district, address_line, nightly_price, cleaning_fee, capacity, bedroom_count, bathroom_count
- booking_id → property_id, guest_id, booking_date, check_in_date, check_out_date, guest_count, total_price, booking_status
- payment_id → booking_id, payment_date, amount, payment_method, payment_status
- review_id → booking_id, rating, comment, review_date
- amenity_id → amenity_name, amenity_category

Transitive dependencies identified in UNF:

- booking_id → guest_id → guest_name, guest_email (transitive through guest_id)
- booking_id → property_id → host_id → host_name, host_email (transitive through property_id and host_id)

5. Normalization Process

Normalization is the process of structuring a relational database to reduce redundancy and improve data integrity. The UNF table is normalized through three stages.

5.1 UNF → 1NF (First Normal Form)

Rules:

- All attributes must contain atomic (indivisible) values
- No repeating groups of columns
- Each row must be uniquely identifiable

Actions taken:

- Removed repeating group columns: amenity_1, amenity_2, amenity_3, photo_1, photo_2
- Moved multi-valued attributes to separate tables: PropertyAmenities and PropertyPhotos
- Each cell now contains a single atomic value

Result: All attributes are atomic, all rows are unique, and repeating groups are eliminated. Data is in 1NF.

5.2 1NF → 2NF (Second Normal Form)

Rules:

- Table must already be in 1NF
- All non-key attributes must be fully dependent on the entire primary key (no partial dependencies)

Actions taken:

- Separated guest information into Guests table (dependent on guest_id)
- Separated property information into Properties table (dependent on property_id)
- Separated host information into Hosts table (dependent on host_id)
- Separated user information into Users table (dependent on user_id)
- Separated payment and review data into their own tables

Result: Each non-key attribute is fully dependent on the primary key of its table. Data is in 2NF.

5.3 2NF → 3NF (Third Normal Form)

Rules:

- Table must already be in 2NF
- No transitive dependencies — non-key attributes must not depend on other non-key attributes

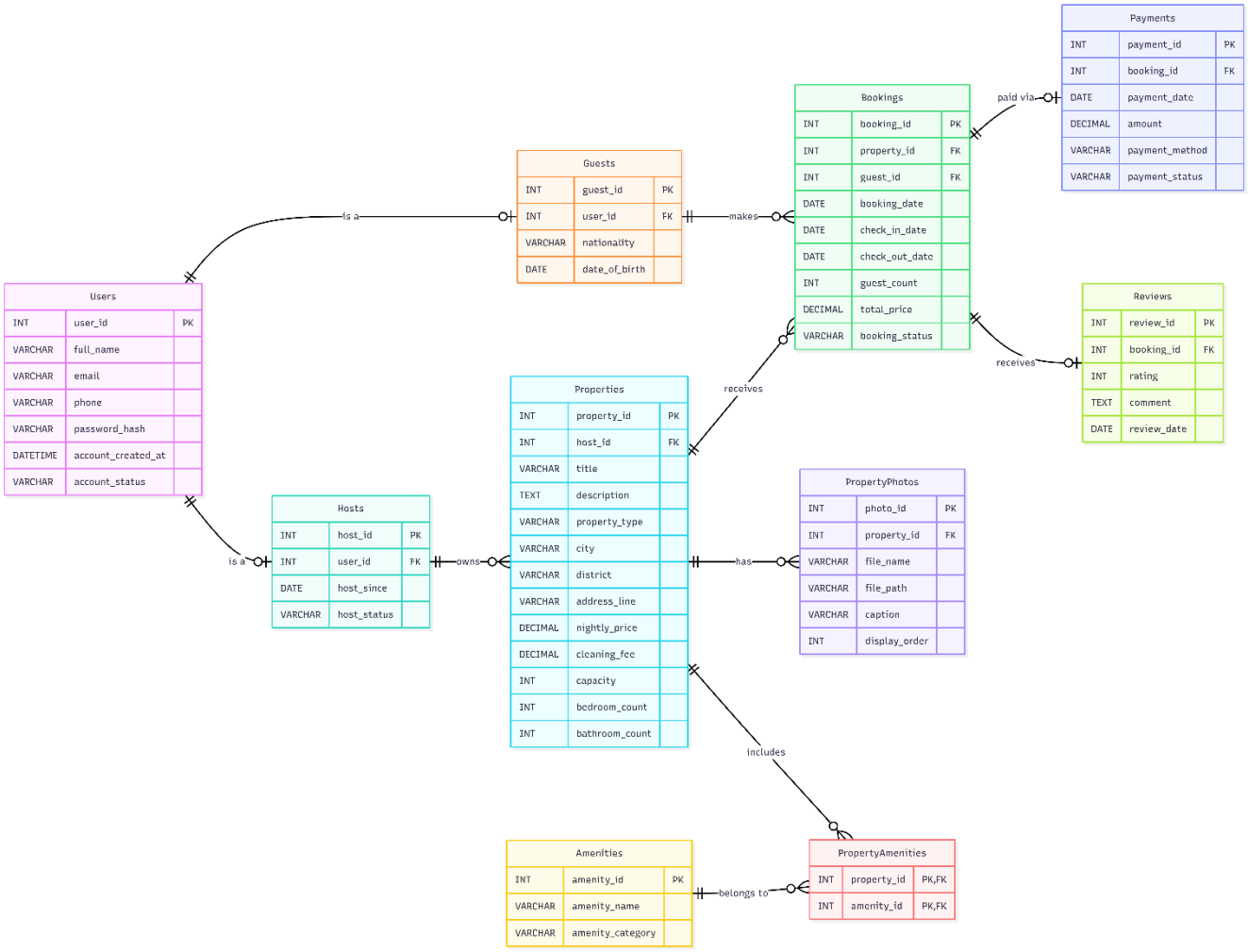
Actions taken:

- Host personal details (name, email, phone) stored only in the Users table, referenced via user_id in Hosts
- Guest personal details stored only in the Users table, referenced via user_id in Guests
- All non-key attributes now depend directly and only on the primary key

Result: No transitive dependencies remain. Data is in 3NF.

6. E-R Diagram

The following diagram illustrates the entity-relationship structure of the 3NF database. Cardinality notation follows the participation format (minimum..maximum).



7. Final 3NF Tables

After normalization, the database consists of 10 tables in Third Normal Form:

Table	Primary Key / Foreign Keys	# Columns
Users	PK: user_id	7
Hosts	PK: host_id | FK: user_id → Users	4
Guests	PK: guest_id | FK: user_id → Users	4
Properties	PK: property_id | FK: host_id → Hosts	13
PropertyPhotos	PK: photo_id | FK: property_id → Properties	6
Bookings	PK: booking_id | FK: property_id → Properties, guest_id → Guests	9
Payments	PK: payment_id | FK: booking_id → Bookings	6
Reviews	PK: review_id | FK: booking_id → Bookings	5
Amenities	PK: amenity_id	3
PropertyAmenities	Composite PK: (property_id, amenity_id) | both are FKs	2

8. SQL Implementation

8.1 Create UNF Table

```sql
CREATE TABLE unf (
    booking_id        INT,
    booking_date      DATE,
    check_in_date     DATE,
    check_out_date    DATE,
    guest_count      INT,
    total_price       DECIMAL(10,2),
    booking_status    VARCHAR(20),
    guest_id          INT,
    guest_name        VARCHAR(100),
    guest_email       VARCHAR(150),
    guest_phone       VARCHAR(20),
    guest_nationality VARCHAR(50),
    guest_dob        DATE,
    property_id       INT,
    property_title    VARCHAR(200),
    property_type     VARCHAR(50),
    city              VARCHAR(100),
    district          VARCHAR(100),
    address_line      VARCHAR(255),
    nightly_price     DECIMAL(10,2),
    cleaning_fee      DECIMAL(10,2),
    capacity          INT,
    bedroom_count     INT,
    bathroom_count    INT,
    host_id           INT,
    host_name         VARCHAR(100),
    host_email        VARCHAR(150),
    host_phone        VARCHAR(20),
    host_since        DATE,
    amenity_1         VARCHAR(100),
    amenity_2         VARCHAR(100),
    amenity_3         VARCHAR(100),
    photo_1           VARCHAR(255),
    photo_2           VARCHAR(255),
    payment_method    VARCHAR(30),
    payment_amount    DECIMAL(10,2),
    payment_status    VARCHAR(20),
    review_rating     INT,
    review_comment    TEXT
);
```

8.2 Insert Sample Data into UNF

The UNF table is populated with 15 rows of sample data representing bookings across 5 properties, 10 guests, and 4 hosts. Each row contains all booking, guest, property, host, amenity, photo, payment, and review data in denormalized form. Full INSERT statements are provided in the supplementary SQL file.

8.3 Create 3NF Tables

```
-- Users Table
CREATE TABLE Users (
    user_id             INT PRIMARY KEY,
    full_name           VARCHAR(100) NOT NULL,
    email               VARCHAR(150) UNIQUE NOT NULL,
    phone               VARCHAR(20),
    password_hash       VARCHAR(255) NOT NULL,
    account_created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    account_status      VARCHAR(20) DEFAULT 'active'
);

-- Hosts Table
CREATE TABLE Hosts (
    host_id     INT PRIMARY KEY,
    user_id     INT NOT NULL,
    host_since  DATE NOT NULL,
    host_status VARCHAR(20) DEFAULT 'active',
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- Guests Table
CREATE TABLE Guests (
    guest_id      INT PRIMARY KEY,
    user_id       INT NOT NULL,
    nationality   VARCHAR(50),
    date_of_birth DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- Properties Table
CREATE TABLE Properties (
    property_id     INT PRIMARY KEY,
    host_id         INT NOT NULL,
    title           VARCHAR(200) NOT NULL,
    description     TEXT,
    property_type   VARCHAR(50),
    city            VARCHAR(100),
    district        VARCHAR(100),
    address_line    VARCHAR(255),
    nightly_price   DECIMAL(10,2) NOT NULL,
    cleaning_fee    DECIMAL(10,2) DEFAULT 0.00,
    capacity        INT,
    bedroom_count   INT,
    bathroom_count INT,
    FOREIGN KEY (host_id) REFERENCES Hosts(host_id)
);

-- PropertyPhotos Table
CREATE TABLE PropertyPhotos (
    photo_id      INT PRIMARY KEY,
    property_id   INT NOT NULL,
    file_name     VARCHAR(255) NOT NULL,
    file_path     VARCHAR(500),
```

```
    caption       VARCHAR(200),
    display_order INT DEFAULT 0,
    FOREIGN KEY (property_id) REFERENCES Properties(property_id)
);

-- Amenities Table
CREATE TABLE Amenities (
    amenity_id       INT PRIMARY KEY,
    amenity_name     VARCHAR(100) NOT NULL,
    amenity_category VARCHAR(50)
);

-- PropertyAmenities Junction Table
CREATE TABLE PropertyAmenities (
    property_id INT,
    amenity_id  INT,
    PRIMARY KEY (property_id, amenity_id),
    FOREIGN KEY (property_id) REFERENCES Properties(property_id),
    FOREIGN KEY (amenity_id) REFERENCES Amenities(amenity_id)
);

-- Bookings Table
CREATE TABLE Bookings (
    booking_id     INT PRIMARY KEY,
    property_id    INT NOT NULL,
    guest_id       INT NOT NULL,
    booking_date   DATE NOT NULL,
    check_in_date  DATE NOT NULL,
    check_out_date DATE NOT NULL,
    guest_count    INT,
    total_price    DECIMAL(10,2),
    booking_status VARCHAR(20) DEFAULT 'confirmed',
    FOREIGN KEY (property_id) REFERENCES Properties(property_id),
    FOREIGN KEY (guest_id) REFERENCES Guests(guest_id)
);

-- Payments Table
CREATE TABLE Payments (
    payment_id     INT PRIMARY KEY,
    booking_id     INT NOT NULL,
    payment_date   DATE NOT NULL,
    amount         DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(30),
    payment_status VARCHAR(20) DEFAULT 'pending',
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
);

-- Reviews Table
CREATE TABLE Reviews (
    review_id   INT PRIMARY KEY,
    booking_id  INT NOT NULL,
    rating      INT CHECK (rating BETWEEN 1 AND 5),
    comment     TEXT,
    review_date DATE NOT NULL,
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
);
```

8.4 Data Migration (UNF → 3NF)

```
-- Step 1: Populate Users (guests)
INSERT INTO Users (user_id, full_name, email, phone, password_hash, account_status)
SELECT DISTINCT guest_id + 100, guest_name, guest_email, guest_phone,
       MD5(guest_email), 'active' FROM unf;

-- Step 1b: Populate Users (hosts, skip duplicates)
INSERT INTO Users (user_id, full_name, email, phone, password_hash, account_status)
SELECT DISTINCT host_id + 200, host_name, host_email, host_phone,
       MD5(host_email), 'active' FROM unf
WHERE host_email NOT IN (SELECT email FROM Users);

-- Step 2: Populate Hosts
INSERT INTO Hosts (host_id, user_id, host_since, host_status)
SELECT DISTINCT host_id, host_id + 200, host_since, 'active' FROM unf;
```

```
-- Step 3: Populate Guests
INSERT INTO Guests (guest_id, user_id, nationality, date_of_birth)
SELECT DISTINCT guest_id, guest_id + 100, guest_nationality, guest_dob FROM unf;

-- Step 4: Populate Properties
INSERT INTO Properties (property_id, host_id, title, property_type,
                        city, district, address_line, nightly_price,
                        cleaning_fee, capacity, bedroom_count, bathroom_count)
SELECT DISTINCT property_id, host_id, property_title, property_type,
       city, district, address_line, nightly_price,
       cleaning_fee, capacity, bedroom_count, bathroom_count FROM unf;

-- Step 5: Populate Amenities
INSERT INTO Amenities (amenity_id, amenity_name, amenity_category) VALUES
(1,'Wi-Fi','basic'),(2,'Air Conditioning','comfort'),(3,'Kitchen','basic'),
(4,'Pool','outdoor'),(5,'Parking','basic'),(6,'Fireplace','comfort'),
(7,'Hiking Trail','outdoor'),(8,'Heating','basic'),(9,'TV','comfort'),
(10,'Jacuzzi','comfort'),(11,'City View','comfort');

-- Step 6: Populate PropertyAmenities
INSERT INTO PropertyAmenities (property_id, amenity_id)
SELECT DISTINCT property_id, 1 FROM unf
WHERE amenity_1='Wi-Fi' OR amenity_2='Wi-Fi' OR amenity_3='Wi-Fi';
-- (repeat for each amenity...)

-- Step 7: Populate Bookings
INSERT INTO Bookings (booking_id, property_id, guest_id, booking_date,
                      check_in_date, check_out_date, guest_count,
                      total_price, booking_status)
SELECT booking_id, property_id, guest_id, booking_date,
       check_in_date, check_out_date, guest_count,
       total_price, booking_status FROM unf;

-- Step 8: Populate Payments
INSERT INTO Payments (payment_id, booking_id, payment_date, amount,
                      payment_method, payment_status)
SELECT booking_id, booking_id, booking_date, payment_amount,
       payment_method, payment_status FROM unf
WHERE payment_amount IS NOT NULL;

-- Step 9: Populate Reviews
INSERT INTO Reviews (review_id, booking_id, rating, comment, review_date)
SELECT booking_id, booking_id, review_rating, review_comment, check_out_date
FROM unf WHERE review_rating IS NOT NULL;
```

9. View: Reconstructing UNF from 3NF

The following VIEW reconstructs the original flat UNF dataset by joining all 10 normalized tables. LEFT JOINs are used for Payments and Reviews since not all bookings have a payment or review record.

```
CREATE VIEW vw_unf_reconstructed AS
SELECT
    b.booking_id, b.booking_date, b.check_in_date, b.check_out_date,
    b.guest_count, b.total_price, b.booking_status,
    g.guest_id,
    ug.full_name     AS guest_name,
    ug.email         AS guest_email,
    ug.phone         AS guest_phone,
    g.nationality    AS guest_nationality,
    g.date_of_birth AS guest_dob,
    p.property_id,
    p.title          AS property_title,
    p.property_type, p.city, p.district, p.address_line,
    p.nightly_price, p.cleaning_fee, p.capacity,
    p.bedroom_count, p.bathroom_count,
    h.host_id,
    uh.full_name     AS host_name,
    uh.email         AS host_email,
    uh.phone         AS host_phone,
    h.host_since,
    pay.payment_method,
    pay.amount       AS payment_amount,
    pay.payment_status,
    rev.rating       AS review_rating,
```

```
    rev.comment      AS review_comment
FROM Bookings b
    JOIN Guests g          ON b.guest_id    = g.guest_id
    JOIN Users ug          ON g.user_id     = ug.user_id
    JOIN Properties p      ON b.property_id = p.property_id
    JOIN Hosts h           ON p.host_id     = h.host_id
    JOIN Users uh          ON h.user_id     = uh.user_id
    LEFT JOIN Payments pay ON b.booking_id  = pay.booking_id
    LEFT JOIN Reviews rev  ON b.booking_id  = rev.booking_id;
```

Running SELECT * FROM vw_unf_reconstructed returns a result set that closely mirrors the original UNF table, confirming the integrity of the normalization process.

10. Conclusion

By transforming an initial unnormalized dataset of 39 columns into 10 well-structured 3NF tables, this project demonstrates the practical importance of database normalization.

Key benefits achieved:

- Eliminated redundancy — host and property details are stored once, not repeated per booking
- Prevented anomalies — updates, insertions, and deletions are localized to a single table
- Enforced integrity — foreign key constraints ensure valid relationships across all tables
- Improved scalability — new entities and relationships can be added without restructuring existing tables
- Enabled flexible querying — normalized tables support diverse queries through JOINs